Smart Canteen Order and Stock Management

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Develop a Python program to process successive customer orders, calculate bills, update available inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

2. Objectives

- Process multiple orders one after another.
- Calculate item totals and order bills.
- Check stock before accepting a quantity.
- Update inventory only for accepted items.
- Display daily sales, rejected items, and remaining stock.

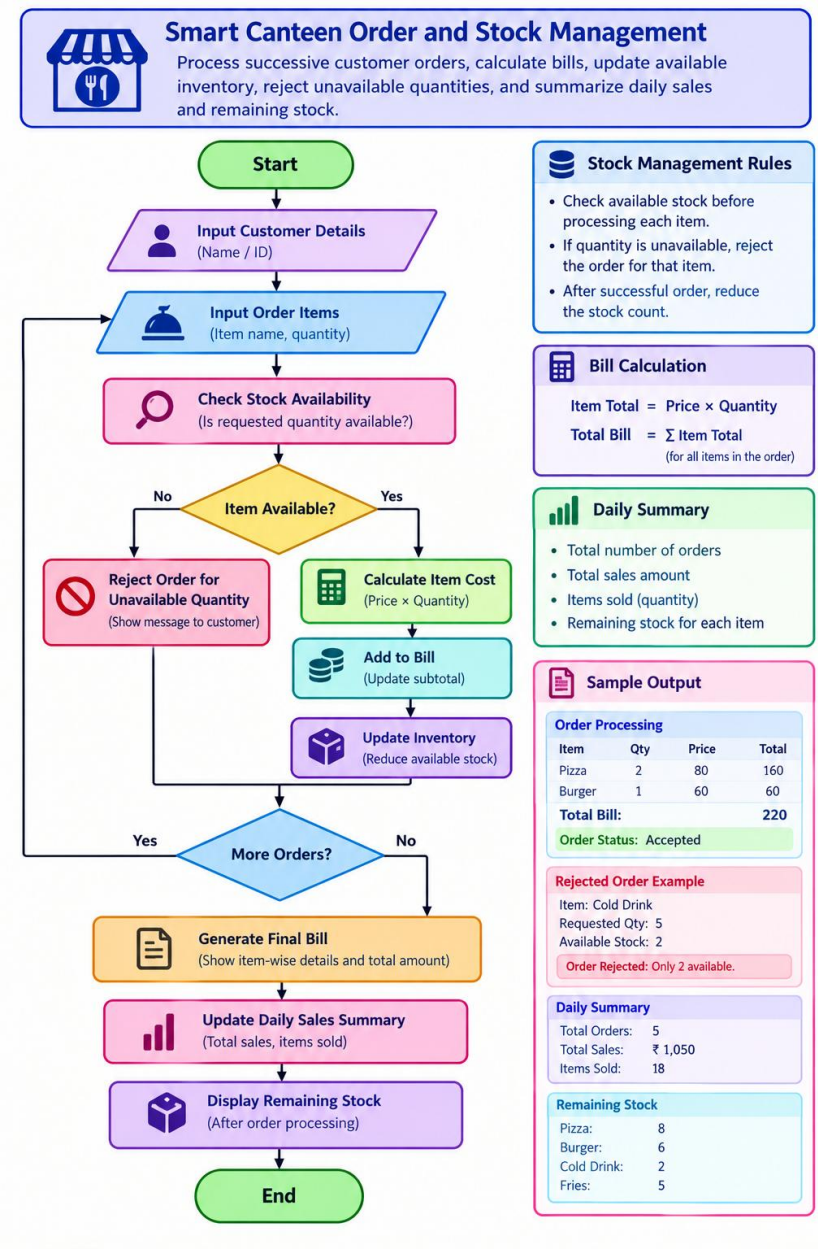
3. Assumptions

The canteen uses a fixed menu and fixed prices. Each order contains item names and quantities. An item is accepted only if its quantity is positive and does not exceed current stock. Rejected items do not affect the bill or inventory. Currency is shown in Indian rupees (Rs.).

4. Algorithm

1. Initialize menu items with price and available stock.
2. Set total daily sales and counters to zero.
3. Read each customer order in sequence.
4. For each item, check whether it exists and quantity is valid.
5. If stock is insufficient, reject the item without changing stock.
6. Otherwise calculate cost, subtract stock, and add cost to bill.
7. Add accepted order bill to daily sales.
8. Print sales summary and remaining stock.

5. Flowchart



6. Pseudocode

BEGIN

```
// Initialize Inventory
stock["Pizza"] = 10
```

```
stock["Burger"] = 8
stock["Cold Drink"] = 5
stock["Fries"] = 10

price["Pizza"] = 80
price["Burger"] = 60
price["Cold Drink"] = 30
price["Fries"] = 40

total_sales = 0
total_orders = 0
total_items_sold = 0

INPUT number_of_orders

FOR order = 1 TO number_of_orders DO

   DISPLAY "Enter Customer Order"

   order_bill = 0
   order_items = 0

   INPUT number_of_items

   FOR i = 1 TO number_of_items DO
```

```
INPUT item_name
INPUT quantity

IF item_name NOT IN stock THEN
   DISPLAY "Invalid Item"
   CONTINUE
END IF

IF quantity <= 0 THEN
   DISPLAY "Invalid Quantity"
   CONTINUE
END IF

IF quantity <= stock[item_name] THEN

   item_total = price[item_name] * quantity

   order_bill = order_bill + item_total

   // Update Inventory
   stock[item_name] =
      stock[item_name] - quantity

   order_items = order_items + quantity

   DISPLAY "Item Added Successfully"
```

```
            DISPLAY item_name
            DISPLAY item_total

        ELSE

            DISPLAY "Order Rejected"
            DISPLAY "Insufficient Stock Available"
            DISPLAY "Available Stock:", stock[item_name]

        END IF

    END FOR

    DISPLAY "Customer Bill:", order_bill

    total_sales = total_sales + order_bill
    total_orders = total_orders + 1
    total_items_sold = total_items_sold + order_items

END FOR

// Generate Daily Sales Summary
DISPLAY "===== DAILY SALES SUMMARY ====="

DISPLAY "Total Orders:", total_orders
DISPLAY "Total Sales:", total_sales
```

```
    DISPLAY "Total Items Sold:", total_items_sold

    // Display Remaining Stock
    DISPLAY "===== REMAINING STOCK ====="

    FOR each item IN stock DO
        DISPLAY item, stock[item]
    END FOR

    DISPLAY "Canteen Management Completed"

END
```

7. Python Source Code

```
# Smart Canteen Order and Stock Management

# Initial inventory and item prices
stock = {
    "Pizza": 10,
    "Burger": 8,
    "Cold Drink": 5,
    "Fries": 10
}

prices = {
    "Pizza": 80,
```

```
    "Burger": 60,
    "Cold Drink": 30,
    "Fries": 40
}

# Daily summary variables
total_sales = 0
total_orders = 0
total_items_sold = 0


def display_menu():
    print("\n========== CANTEEN MENU ==========")
    print(f"{'Item':<15}{'Price (Rs.)':<15}{'Stock'}")
    print("-" * 40)

    for item in stock:
        print(f"{item:<15}{prices[item]:<15}{stock[item]}")

    print("=" * 40)


def process_order():
    global total_sales, total_orders, total_items_sold

    customer_name = input("\nEnter customer name: ")
```

```
order_bill = 0
order_items = 0

display_menu()

try:
    number_of_items = int(input("Number of different items: "))
    if number_of_items <= 0:
        print("Invalid number of items.")
        return
except ValueError:
    print("Please enter a valid number.")
    return

for i in range(number_of_items):
    print(f"\nItem {i + 1}")
    item_name = input("Enter item name: ").strip().title()

    if item_name not in stock:
        print("Item not available in the menu. Order rejected.")
        continue

    try:
        quantity = int(input("Enter quantity: "))
        if quantity <= 0:
            print("Invalid quantity. Please enter a positive number.")
```

```
            continue
        except ValueError:
            print("Invalid quantity. Please enter a whole number.")
            continue

        # Check stock availability
        if quantity > stock[item_name]:
            print("Order rejected: Insufficient stock!")
            print("Available stock:", stock[item_name])
            continue

        # Calculate item bill
        item_total = prices[item_name] * quantity
        order_bill += item_total
        order_items += quantity

        # Update inventory
        stock[item_name] -= quantity

        print("Item accepted:", item_name)
        print("Item total: Rs.", item_total)
        print("Remaining stock:", stock[item_name])

    # Complete customer order
    print("\n========== CUSTOMER BILL ==========")
    print("Customer:", customer_name)
```

```
    print("Total items sold:", order_items)
    print(f"Total bill: Rs. {order_bill:.2f}")
    print("==================================")

    total_sales += order_bill
    total_orders += 1
    total_items_sold += order_items


def daily_summary():
    print("\n========== DAILY SALES SUMMARY ==========")
    print("Total orders processed:", total_orders)
    print(f"Total sales: Rs. {total_sales:.2f}")
    print("Total items sold:", total_items_sold)

    print("\n========== REMAINING STOCK ==========")
    print(f"{'Item':<20}{'Remaining Quantity'}")
    print("-" * 40)

    for item, quantity in stock.items():
        print(f"{item:<20}{quantity}")

    print("=" * 40)


# Main program
```

```
print("=" * 45)
print(" SMART CANTEEN ORDER AND STOCK MANAGEMENT")
print("=" * 45)

while True:
    print("\n1. Display Menu")
    print("2. Process Customer Order")
    print("3. View Daily Summary")
    print("4. Exit")

    choice = input("Enter your choice (1-4): ").strip()

    if choice == "1":
        display_menu()
    elif choice == "2":
        process_order()
    elif choice == "3":
        daily_summary()
    elif choice == "4":
        daily_summary()
        print("Thank you! Canteen management completed.")
        break
    else:
        print("Invalid choice. Please select 1, 2, 3, or 4.")
```

8. Output Screenshot

```
==========================================
 SMART CANTEEN ORDER AND STOCK MANAGEMENT
==========================================

1. Display Menu
2. Process Customer Order
3. View Daily Summary
4. Exit
Enter your choice (1-4): 2

Enter customer name: Lekhana

========== CANTEEN MENU ==========
Item           Price (Rs.)     Stock
--------------------------------------
Pizza          80              10
Burger         60              8
Cold Drink     30              5
Fries          40              10
======================================
Number of different items: 2

Item 1
Enter item name: Pizza
Enter quantity: 2
Item accepted: Pizza
Item total: Rs. 160
Remaining stock: 8

Item 2
Enter item name: Burger
Enter quantity: 1
Item accepted: Burger
Item total: Rs. 60
Remaining stock: 7

========== CUSTOMER BILL ==========
Customer: Lekhana
Total items sold: 3
Total bill: Rs. 220.00
==================================
```

9. Test Cases

1. **TC01 – Display Menu:** Check whether all canteen items, prices, and available stock are displayed correctly.

2. **TC02 – Successful Order:** Order 2 Pizzas and verify that the bill is ₹160 and the stock decreases from 10 to 8.

3. **TC03 – Multiple Items Order:** Order 2 Pizzas and 1 Burger and verify that the total bill is ₹220.

4. **TC04 – Exact Available Stock:** Order 8 Burgers and verify that the order is accepted and the remaining stock becomes 0.

5. **TC05 – Insufficient Stock:** Order 6 Cold Drinks when only 5 are available and verify that the order is rejected.

6. **TC06 – Invalid Item:** Enter an unavailable item name and verify that an error message is displayed.

7. **TC07 – Zero Quantity:** Enter quantity 0 and verify that the order is rejected.

8. **TC08 – Negative Quantity:** Enter a negative quantity and verify that an invalid quantity message is displayed.

9. **TC09 – Invalid Menu Choice:** Enter an invalid menu option and verify that an error message is displayed.

10. **TC10 – Daily Sales Summary:** Verify that total orders, total sales, items sold, and remaining stock are displayed correctly.

11. **TC11 – Multiple Customer Orders:** Process successive customer orders and verify that sales and inventory are updated cumulatively.

12. **TC12 – Exit Program:** Select option 4 and verify that the daily summary is displayed and the program terminates correctly.

10. Conclusion

The program processes multiple canteen orders, calculates bills, rejects requests that exceed available stock, updates inventory for accepted items, and reports daily sales and remaining stock.